

SPEC 2111

Deep Learning and Neural Networks

Course Roadmap

PART-1: Introduction, Foundations & Essentials

- Week 1 – Introduction & Course Overview
- **Week 2 – Linear Regression** 
 - Regression concepts, gradient descent, loss functions
- Week 3 – Logistic Regression
 - Classification, sigmoid, cross-entropy loss

PART-2: Neural Networks, Deep Learning and Training

- Week 4 – Neural Network Essentials
 - Neurons, layers, forward propagation, activation functions
- Week 5 – Refining Backpropagation and Deep Neural Networks (DNNs)
 - Backpropagation, chain rule, gradient descent optimization, intro to multiple layers
- Week 6 – Preventing Overfitting
 - Overfitting vs underfitting, regularization (L1/L2), dropout, early stopping

PART-3: Specialized Architectures

- Week 7 – Convolutional Neural Networks (CNNs)
 - Convolutions, pooling, image applications
 - **Assessment-1**
- Week 8 – Representations & Transfer Learning (and Test)
 - Feature learning, pre-trained models, fine-tuning
- Week 9 – Recurrent Neural Networks & LSTM
 - RNNs, LSTM/GRU, sequence modeling
- Week 10 – Attention & Transformers
 - Attention mechanism, transformer architecture, NLP applications

PART-4: Reinforcement Learning & Projects

- Week 11 – Reinforcement Learning
 - **Assessment-2**
- Week 12 - Project Presentations & Discussions (**Project Due**)

Week-2 / Agenda

Theme: LINEAR REGRESSION

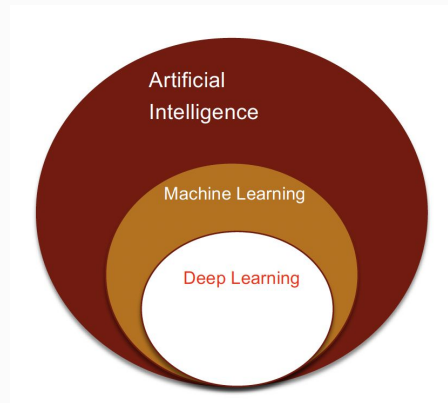
- Theory
 - Recap - Week 1
 - Components of learning
 - Linear regression as a learning model
 - Hypothesis function and parameters
 - Measuring error: cost functions
 - Learning as optimization
 - Gradient descent
 - Scaling, normalization, and convergence
 - From linear regression to neural networks
- Lab Work



Recap

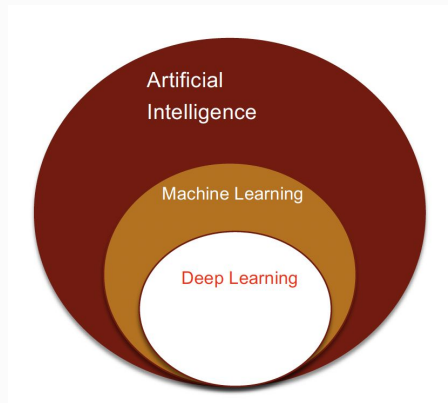
- AI $\rightarrow$ ML $\rightarrow$ DL -

- How we moved from
 - Traditional rule based AI to
 - hard to manually craft rules in complex domains
 - Machine learning to
 - finding good feature representations is not trivial and takes time
 - Deep Learning



Recap

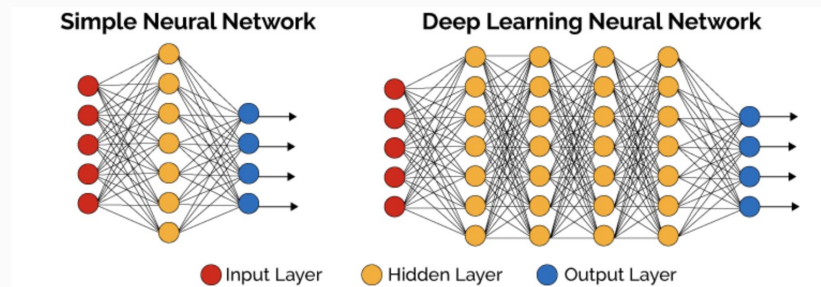
- Why Deep Learning Works Now -
 - Emerged from the confluence of three forces:
 - New algorithmic ideas → improved learning techniques for neural networks
 - Lots of data → huge datasets for training
 - Parallel computing hardware → GPUs for fast computation
 - These forces together revived an old concept: neural networks



Recap

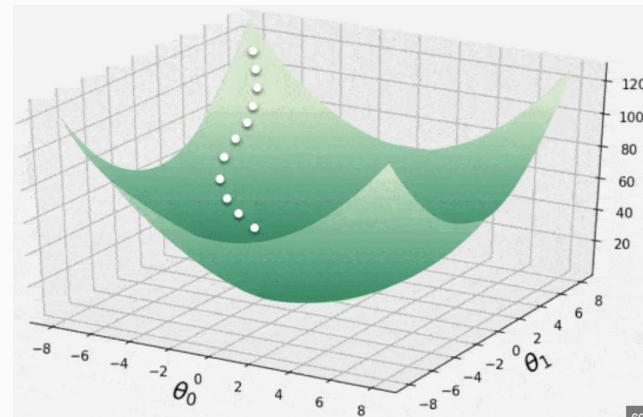
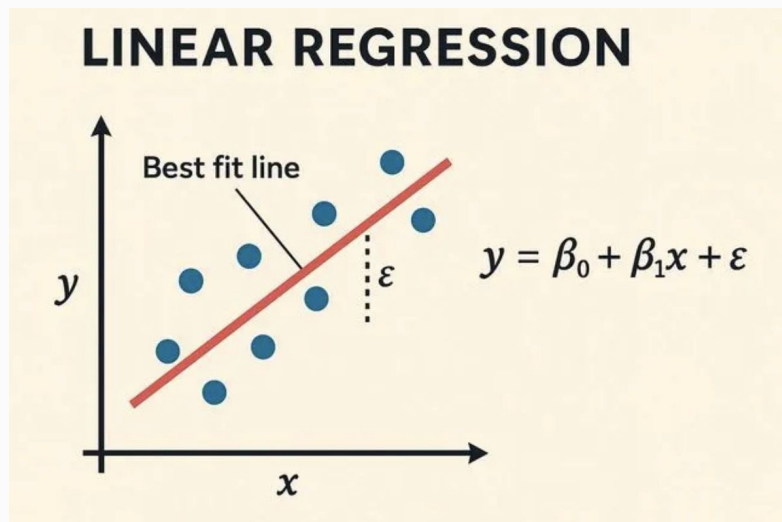
- NN to DL -

- Neural Network
 - made of neurons organized in input → hidden → output
 - typically only one hidden layer.
- Deep Learning
 - Neural networks with multiple hidden layers - hence deep!
 - These layers allow the network to learn hierarchical features automatically



Foundations of Neural Networks and Deep Learning

Linear Regression and Optimization



Learning is Finding Patterns in the Data

- Learning = finding patterns in data
- Goal: predict, generate, or understand
- From a mathematical perspective, learning is an optimization problem
 - searching for the best model in a space that fits the data well.

Main Components of a Learning Algorithm

A learning algorithm consists of:

- A model (with learnable parameters)
 - Function that maps inputs to outputs
 - Contains parameters - values - that we don't know but must learn from the data
- A loss / cost function (measures error)
 - Measures how wrong models predictions compared to ground truth
 - Tells us how poorly a model is performing
- An optimization method (updates parameters)
 - Procedure that tells how to change parameters to reduce the cost
 - Repeatedly update parameters to reduce the cost

Main Components of a Learning Algorithm

A learning algorithm consists of:

1. **A model (with learnable parameters)**
2. **A loss / cost function (measures error)**
3. **An optimization method (updates parameters)**

Learning is the process of finding the parameter values that minimize a cost function.

Now that we understand learning as optimization, let's look at the simplest model that actually learns
linear regression

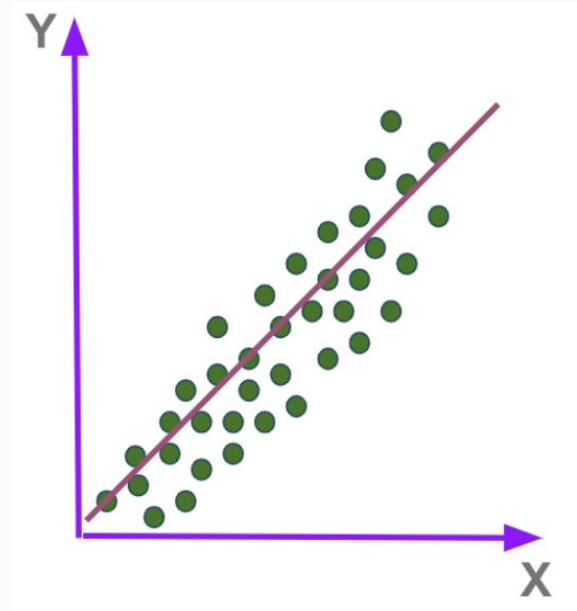
And let's also ground this framework
model, cost function and optimization
with Linear Regression

1. Let's start with the **model** behind Linear Regression and its parameters

Linear Regression (LR)

- Model -

- LR models the relationship between a target and features using a parameterized function
 - function is linear in parameters
- Model: $Y = wX + b$
 - Y is the target variable
 - X is the input variable
- Parameters to find: w, b



Target Variable

- **target** is the output variable that our model aims to predict.
- In statistics, commonly referred to as the **dependent variable**.
- In linear regression, target variable is a continuous & real-valued.
- *Note that linear regression is not intended to be used for categorical or binary prediction tasks.*

$$Y = wX + b$$

Input Variables

- Called also as **predictor features**
- referred to as **independent variables** in the statistics.
- predictor features are usually denoted with the symbol X

$$Y = wX + b$$

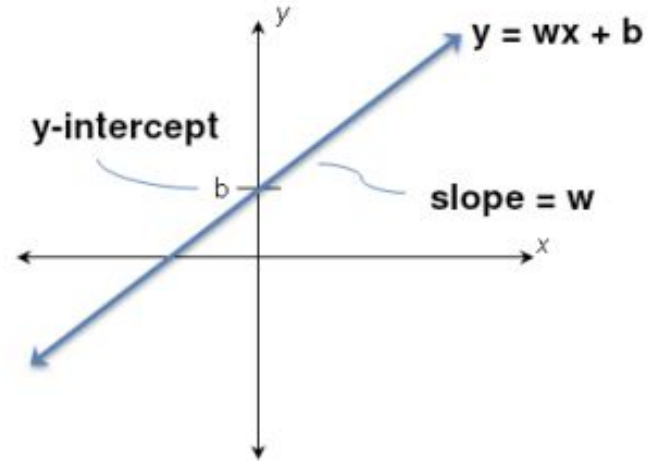
- Single feature $\rightarrow X$
- Multiple features $\rightarrow X_1, X_2, \dots, X_n$
- Each feature captures one aspect of the input data

Components of Linear Regression

- Slope -

- slope controls the steepness/tilt of the line
- determines how much y changes when x increases by 1:
 - Large $|w| \rightarrow$ steep line
 - $w > 0 \rightarrow$ line goes up as x increases
 - $w < 0 \rightarrow$ line goes down as x increases

$$Y = wX + b$$

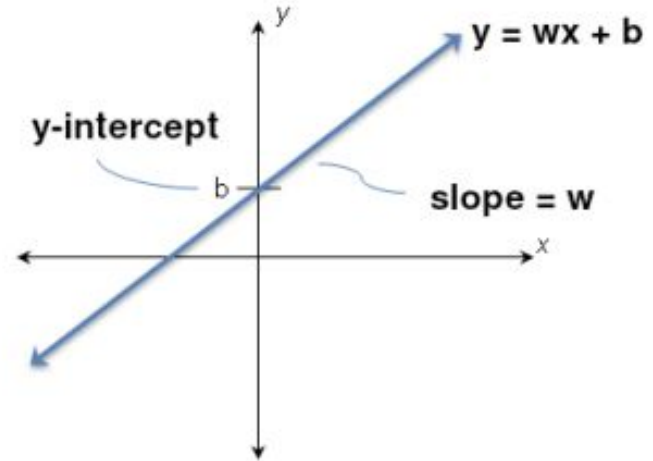


Components of Linear Regression

- Intercept -

- intercept shifts the line up or down along the y-axis:
 - b is the value of y when $x = 0$
 - Increasing b lifts the entire line upward
 - Decreasing b pushes the entire line downward

$$Y = wX + b$$



Let's examine how LR works with An Example Problem

- Goal: predict height using age
- Age → input feature (independent variables)
- Height → target (dependent variable)

$X = \text{age} = [1. 3. 4. 7. 10. 11. 14. 13.5 16. 18. 22.]$

$Y = \text{height} = [60. 90. 100.5 130. 128. 140. 160. 140. 165. 162. 155.]$

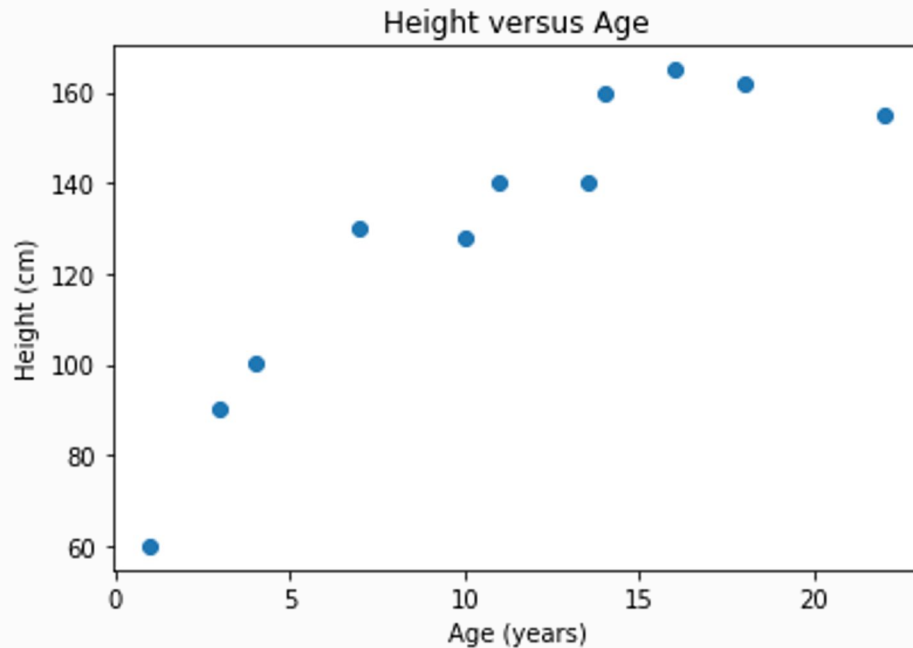
We will use this toy
example throughout
the slides + in Lab-2
notebook.

Lets Visualize Height vs Age

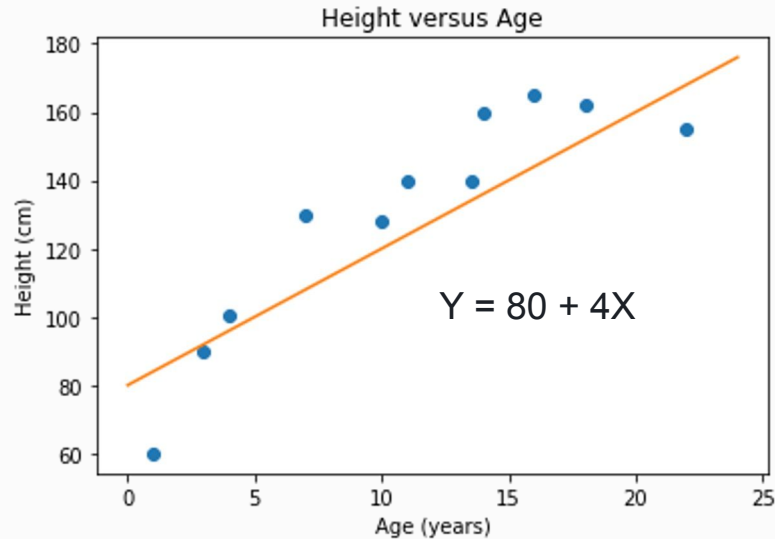
X = age = [1. 3. 4. 7. 10. 11. 14. 13.5 16.
18. 22.]

Y = height = [60. 90. 100.5 130. 128. 140.
160. 140. 165. 162. 155.]

Examining our data we can
clearly see a relationship
between Age and Height



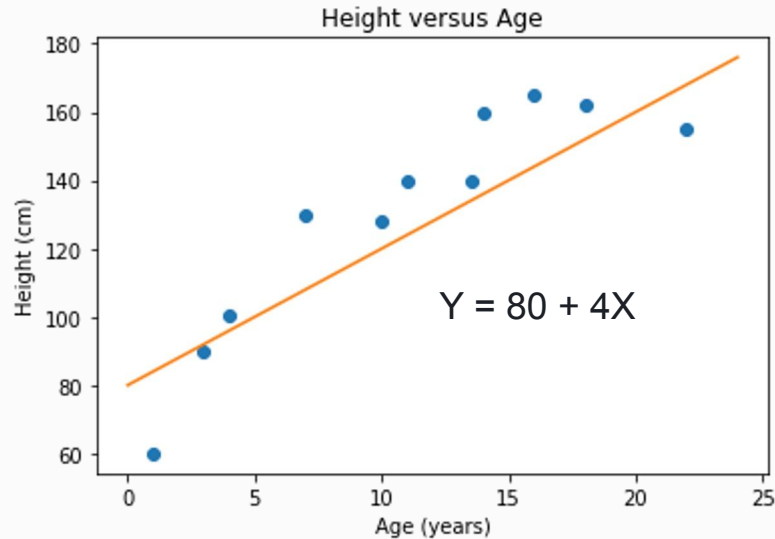
Example Functions Against our Data



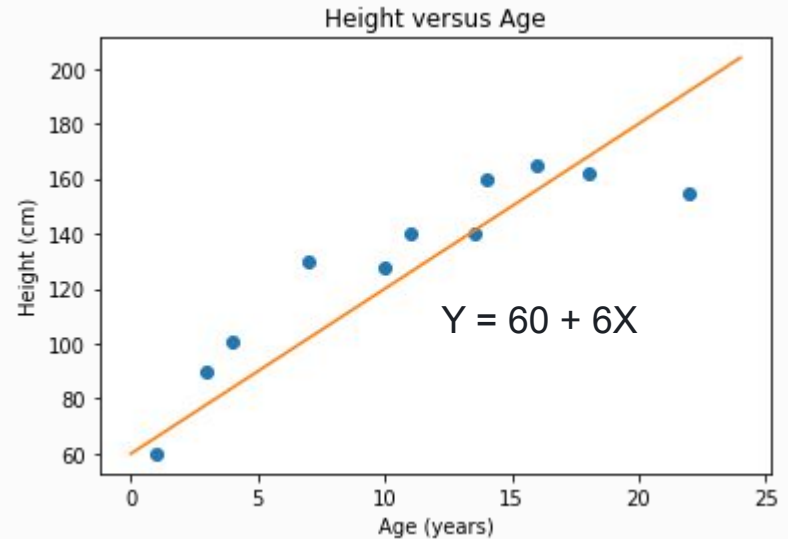
*we might eyeball the data and guess
that a reasonable fit*

We might try $W=4$, $b = 80$

Example Functions Against our Data



*we might eyeball the data and guess
that a reasonable fit
We might try $W=4$, $b = 80$*



*We might think that the linear function
 $W=6$, $b = 60$ might give a better fit.*

Hypothesis Function

- The **hypothesis function** is part of the hypothesis space — the set of all functions the algorithm could choose from to fit the data.
- During training, the algorithm learns parameters (weights) to make the hypothesis function to match the actual outputs as well as possible.
- For linear regression, the hypothesis function is a linear equation that predicts the output from the input(s).

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$h_{\theta}(x)$ = our current guess of the output for input x

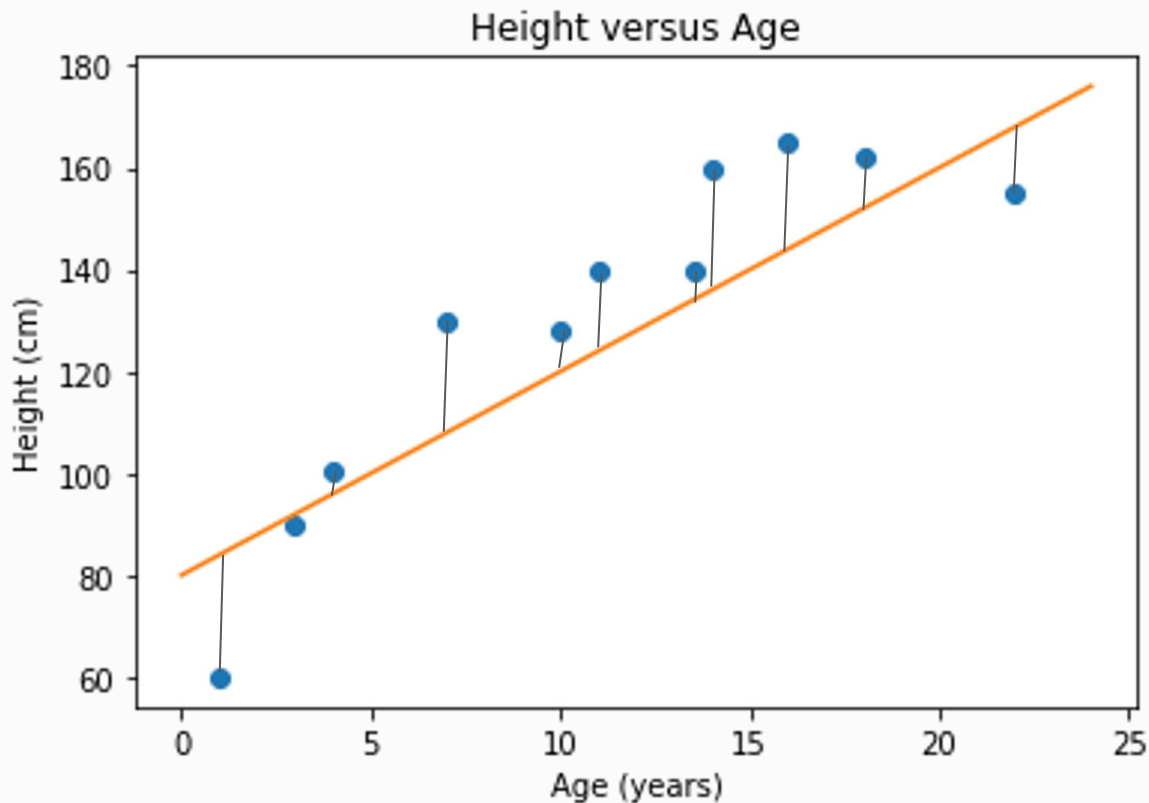
called hypothesis function, because it is the function we are testing or evaluating

shape of the hypothesis is fixed (linear), but we adjust parameters to make it fit the data.

We'll measure how good our hypothesis is and tweak the parameters acc.

1. Let's start with the **model** behind Linear Regression and its parameters
2. Let's talk about the **cost function** that will help us to evaluate one set of parameter values over another
 - a. cost function measures how wrong the model's predictions are compared to the true data.
 - b. gives us a single number that tells us how well or how poorly the model is performing.

Let's start with defining Loss Function



Loss Function (Single Example)

- Measures how wrong the model is for a single data point.
- In LR, we use squared error:

$$L(h_{\theta}(x^{(i)}), y^{(i)}) = (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Key Idea: "How badly did I do on this one example?"

Squared Error as the Loss Function

- Instead of summing the raw differences, we sum the squared differences. This has two benefits:
 - positive and negative errors won't cancel each other.
 - larger errors penalized more heavily, small errors penalized lightly.
 - this encourages model to focus on correcting biggest mistakes first.
 - makes optimisation problem nice for gradient-based methods as squared error produces a smooth, continuous and differentiable cost function.

Loss Function (Single Example)

- Measures how wrong the model is for a single data point.
- In LR, we use squared error:

$$L(h_{\theta}(x^{(i)}), y^{(i)}) = (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Key Idea: "How badly did I do on this one example?"

Cost Function (All Examples / Aggregate)

- Mean Squared Error (MSE) -

- Measures how wrong the model is overall on the dataset.
- Often just the average of the loss over all examples:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m L(h_{\theta}(x^{(i)}), y^{(i)})$$

Key idea: "How badly does my hypothesis perform on the whole dataset?"

Cost Function in Linear Regression

In LR, we use Mean Squared Error (MSE)

For one predictor, cost function becomes:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$


Loss Function

where

- θ_0 and θ_1 are the parameters of our model
- m is the number of training examples
- y^i is the i th target value
- x^i is the i th feature vector, and
- h_{θ} is our model (hypothesis function)

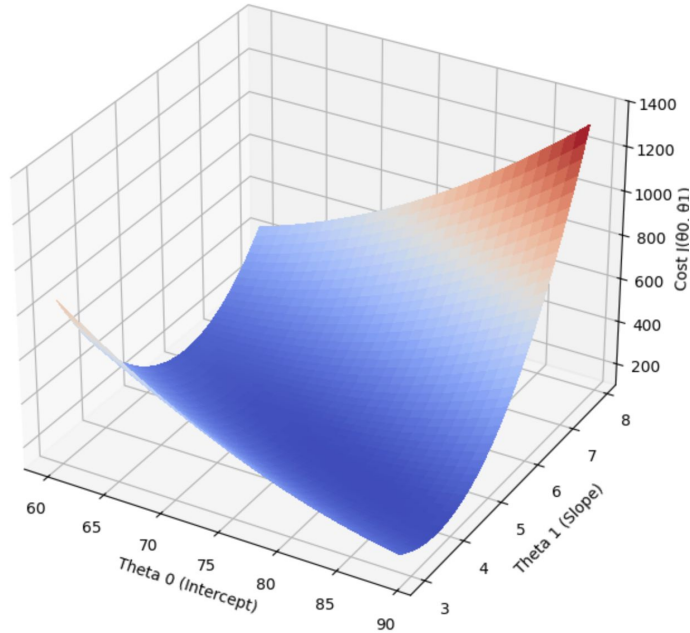

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Cost Function in Linear Regression

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- $\frac{1}{2m}$ is not technically needed for the cost function (in MSE definition)
- simplifies the derivative of the cost function later when we optimize the parameters numerically
- Using $\frac{1}{2m}$ does not change the model you learn or the location of the optimum you find.
 - It's basically just a convention that makes the math tidier during optimisation.

Cost Function Visualization



Plot the surface of the cost function

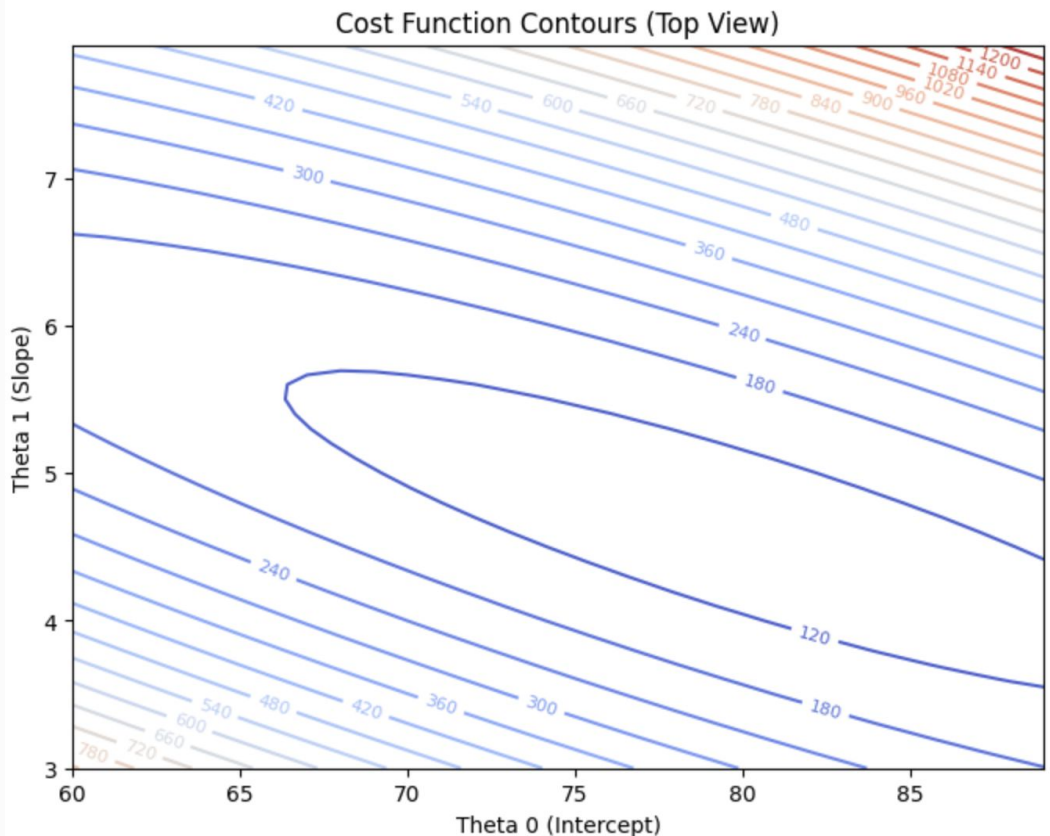
$X = \text{age} = [1. 3. 4. 7. 10. 11. 14. 13.5 16. 18. 22.]$

$Y = \text{height} = [60. 90. 100.5 130. 128. 140. 160. 140. 165. 162. 155.]$

- Each point on this surface corresponds to one specific linear model: $h\theta(x) = \theta_0 + \theta_1 x$
- X axis: θ_0 (intercept)
- Y axis: θ_1 (slope)
- Z axis: $J(\theta_0, \theta_1)$ (cost)

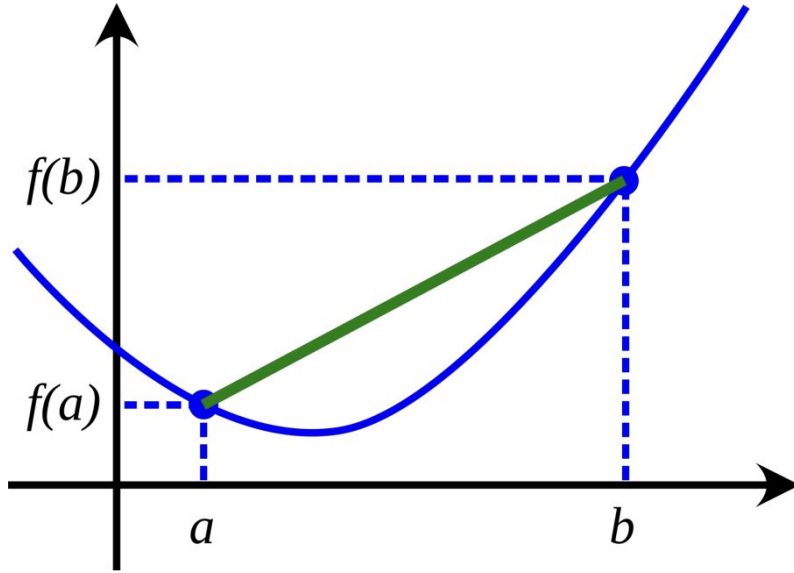
The bottom of the bowl represents the best-fitting line for your data.

Contour plot



- Contours = lines of equal cost
- Cost decreases as you move toward the center
- Cost increases as you move away from the center

Convex Function

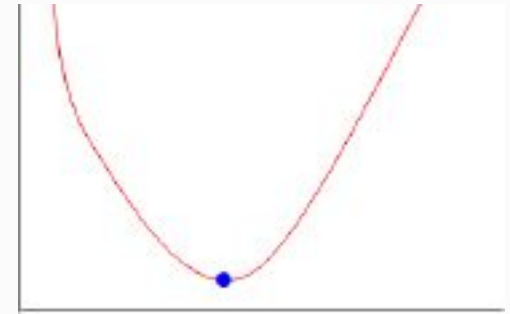


line segment between any two points on the graph of the function lies above the graph itself

- A convex function is a special type of function that has a “bowl-shaped” form
- very nice mathematical properties – especially for optimisation problems
- any local minimum is also the global minimum, so algorithms like gradient descent reliably find the best solution.

Linear Regression has a Convex Cost Surface

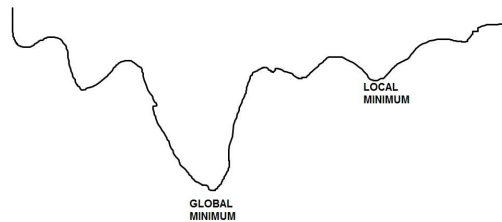
- Since cost function is MSE
- MSE cost is a quadratic function of the parameters, and this means there is a single global minimum.
- An optimizer like gradient descent can reliably find it.
- Another implication → Wherever you start as your initial parameters, if you follow steps that consistently lower the cost, you will end up at the global minimum



Cost Function Surface

Reality Can Be More Complex

- Real-world error surfaces may be non-linear, wavy, multi-dimensional.
- Can contain multiple local minima or saddle points.
- Gradient descent might get stuck in local minima.



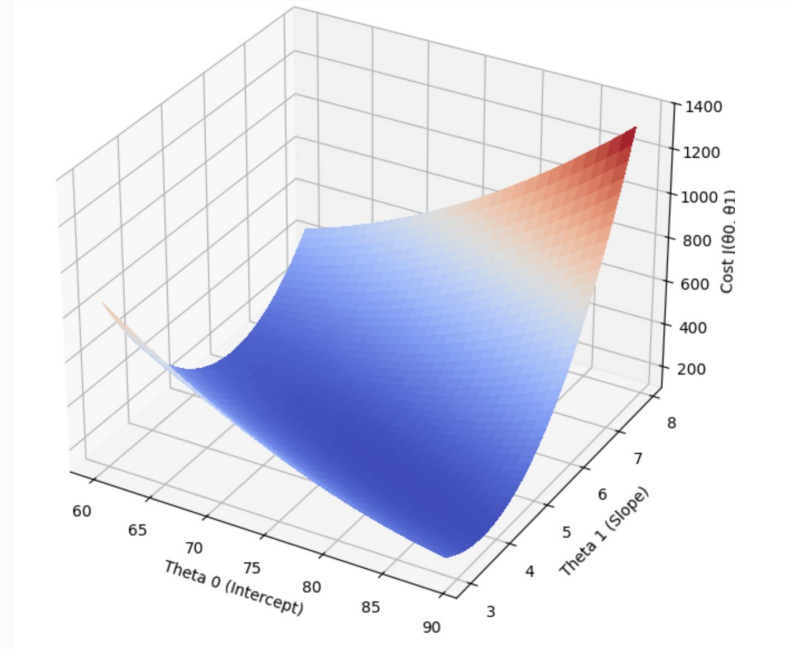
Link to Neural Networks

- Neural networks have complex, non-convex surfaces.
- Use advanced optimizers (Adam, RMSProp) and techniques to navigate them.

1. Let's start with the **model** behind Linear Regression and its parameters
2. Let's talk about the **cost function** that will help us to evaluate one set of parameter values over another
3. Now that we have a way to evaluate different parameter sets, how can we find the one that give the lowest cost? With **optimization procedure**

What is an Optimization Algorithm?

- rather than checking every possible value of θ , we use optimization algorithms that let the mathematics guide us downhill toward the minimum of the cost function.
- most commonly used optimization algorithm for this purpose is **Gradient Descent**.

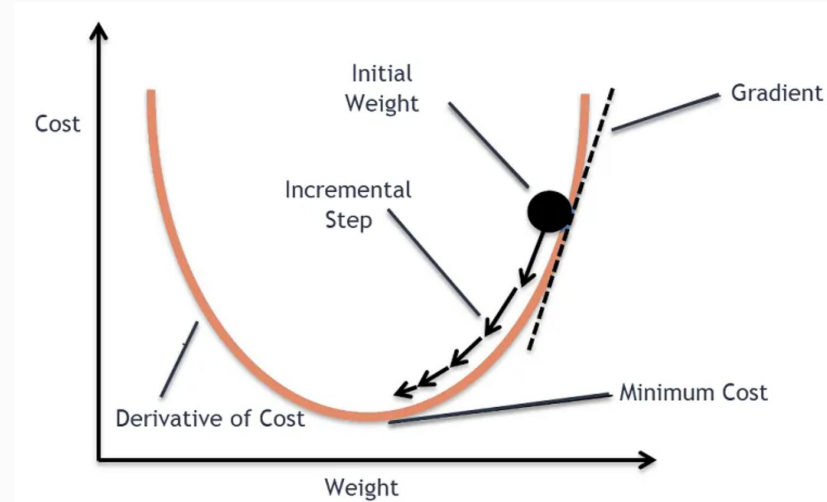


Gradient Descent Algorithm

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Cost Function to Minimize

- Gradient descent updates model parameters to improve predictions → to lower the cost.
- It checks how cost changes when we tweak each parameter.
 - a. This is done by using the derivatives of the model function wrt parameters



$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Cost Function to Minimize

Derivative tells us..

- Direction of Increase
 - derivative tells us which direction makes the cost increase.
 - Positive derivative → cost increases if parameter increases
 - Negative derivative → cost decreases if parameter increases
 - So the derivative gives the wrong direction — and we go the opposite way!!
- Rate of Change (Steepness)
 - derivative also tells us how fast the cost is changing.
 - Large derivative → very steep → take a bigger correction step
 - Small derivative → gentle slope → take a smaller step
 - This helps control how much we move.

Lets see this in action!

Recall the Cost function we had:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Where:

- $h_{\theta}(x) = \theta_0 + \theta_1 x$ (our prediction)
- m = number of data points

Lets see this in action!

Now compute the gradients of the cost with respect to our model parameters:

Cost function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Derivatives wrt to model parameters

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Derivative give us:

- **Direction:** which way to go to increase the cost function (gradient points uphill).
 - We should move reverse!
- **Rate/strength:** how steep the increase is.

We will use these derivatives in gradient descent to decide on the update rate and direction!

Derivative of Cost w.r.t. θ_0

Cost function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Take derivative

$$\frac{\partial J}{\partial \theta_0} = \frac{\partial}{\partial \theta_0} \left[\frac{1}{2m} \sum \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 \right]$$

Move constant outside

$$= \frac{1}{2m} \sum \frac{\partial}{\partial \theta_0} \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Chain rule

$$= \frac{1}{2m} \sum 2 \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_0} h_{\theta}(x^{(i)})$$

Hypothesis

$$h_{\theta}(x) = \theta_0 + \theta_1 x \quad \Rightarrow \quad \frac{\partial}{\partial \theta_0} h_{\theta}(x^{(i)}) = 1$$

Result

$$\boxed{\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)}$$

Derivative of square:

$$\frac{d}{dz}(z^2) = 2z$$

Maths behind derivatives

Derivative of Cost w.r.t. θ_1

Cost function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Take derivative

$$\frac{\partial J}{\partial \theta_1} = \frac{\partial}{\partial \theta_1} \left[\frac{1}{2m} \sum \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 \right]$$

Move constant outside

$$= \frac{1}{2m} \sum \frac{\partial}{\partial \theta_1} \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Chain rule

$$= \frac{1}{2m} \sum 2 \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_1} h_{\theta}(x^{(i)})$$

Hypothesis derivative

$$h_{\theta}(x) = \theta_0 + \theta_1 x \quad \Rightarrow \quad \frac{\partial}{\partial \theta_1} h_{\theta}(x^{(i)}) = x^{(i)}$$

Result

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

Maths behind derivatives

Lets see this in action!

Now update the parameters
using the derivatives

$$\begin{aligned}\theta_0 &:= \theta_0 - \alpha \frac{\partial J}{\partial \theta_0} \\ \theta_1 &:= \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}\end{aligned}$$

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

We will use these derivatives in gradient descent to decide on the update rate and direction!

Lets see this in action!

Now update the parameters using the derivatives

$$\begin{aligned}\theta_0 &:= \theta_0 - \alpha \frac{\partial J}{\partial \theta_0} \\ \theta_1 &:= \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}\end{aligned}$$

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

We will use these derivatives in gradient descent to decide on the update rate and direction!

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^i$$

α is the **learning rate** (step size).

If we moved exactly by the gradient value steps could be too large, we might overshoot the minimum, so learning rate controls the step size

Gradient Descent Algorithm

Initialize

- Choose learning rate α
- Initialize parameters θ_0, θ_1 (often 0 or small random values)

Repeat until convergence

Compute gradients:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x^{(i)}$$

Update simultaneously

$$\theta_0 := \theta_0 - \alpha \cdot \frac{1}{m} \sum (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \cdot \frac{1}{m} \sum (h_{\theta}(x^{(i)}) - y^{(i)})x^{(i)}$$

Updates must be simultaneous
each step uses the gradient with respect to all parameters simultaneously, so the update reflects their combined effect on the cost.

When to Stop Gradient Descent Algorithm

The most common ways to **stop gradient descent** are:

1. Gradient Magnitude:

- Stop when the **overall gradient vector** is very small:

$$\|\nabla_{\theta} J\| < \epsilon$$

- Means the slope is nearly flat in all directions.

2. Cost Function Change:

- Stop when the **change in cost** between steps is tiny:

$$|J(\theta_{t+1}) - J(\theta_t)| < \epsilon$$

- Further updates won't meaningfully reduce the cost.

3. Maximum Iterations / Epochs:

- Stop after a predefined number of steps to prevent infinite loops.

4. Combination / Convergence Criteria:

- Often use both gradient magnitude **and** cost change together for robust stopping.

Rule of thumb: stop when updates don't improve the cost in any meaningful way.

Gradient Descent Algorithm

Input:

- Data points `(x[1..m], y[1..m])`
- Learning rate `α`
- Number of iterations `num_iters`

Initialize parameters:

- `θ0 = 0`
- `θ1 = 0`

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$\theta_0 := \theta_0 - \alpha \frac{\partial J}{\partial \theta_0}$$

$$\theta_1 := \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}$$

```
# Initialize parameters
theta0 = 0.0 # w
theta1 = 0.0 # b

learning_rate = 0.1
max_iters = 10000
epsilon = 1e-6

# MSE cost function
def mse(theta0, theta1, x, y):
    y_pred = theta0 * x + theta1
    return np.mean((y - y_pred)**2)

# Gradients
def gradients(theta0, theta1, x, y):
    m = len(x)
    y_pred = theta0 * x + theta1
    dtheta0 = (-1/m) * np.sum(x * (y - y_pred))
    dtheta1 = (-1/m) * np.sum(y - y_pred)
    return dtheta0, dtheta1

# Gradient Descent Loop
for i in range(max_iters):
    dtheta0, dtheta1 = gradients(theta0, theta1, x, y)

    theta0_new = theta0 - learning_rate * dtheta0
    theta1_new = theta1 - learning_rate * dtheta1

    # Check delta cost
    delta_cost = abs(mse(theta0_new, theta1_new, x, y) - mse(theta0, theta1, x, y))

    theta0, theta1 = theta0_new, theta1_new

    if delta_cost < epsilon:
        print(f"Stopped at iteration {i+1}")
        break
```

How to choose number of iterations?

What does it affect?

- Controls how long gradient descent runs
- Determines how close we get to the minimum cost
- Too few iterations → algorithm stop too early when potentially the cost is still too high, model might underfit
- Too many iteration → usually no harm but increased computational cost unnecessarily
- Enough iterations → cost stabilizes, parameters stop changing much

How to choose number of iterations?

What does it affect?

- Controls how long gradient descent runs
- Determines how close we get to the minimum cost
- Too few iterations → algorithm stop too early when potentially the cost is still too high, model might underfit
- Too many iteration → usually no harm but increased computational cost unnecessarily
- Enough iterations → cost stabilizes, parameters stop changing much

How do we choose or change it?

- Start with a fixed value (e.g., 1000 iterations)
- Check cost difference between current and prev iteration
- If cost is still decreasing → increase iterations
- If cost is flat → stop earlier

What is the Learning Rate?

- Learning rate α controls how big a step we take in each update.
- It determines how fast the model learns.

How Does the Learning Rate Affect Training?

Learning rate too small:

- Very slow learning
- Many iterations needed
- Algorithm converges safely but inefficiently → similar to high number of iterations case

Learning rate too large:

- Cost may oscillate or increase
- Algorithm can overshoot the minimum
- May fail to converge

Learning rate just right:

- Smooth decrease in cost
- Reaches the minimum efficiently

How to Choose Learning Rate?

Start:

- Use $\alpha = 0.01$ (or 0.001 if unsure)

During training, monitor cost change:

$$\Delta J = J_t - J_{t-1}$$

Adjust α as follows:

- Cost decreases smoothly → **keep α**
- Cost increases or oscillates → **α too large → divide α by 10**
- Cost decreases very slowly → **α too small → increase α**

How to choose initial values?

- The most common approach is just to pick a small number, e.g., 0, 0.1, or 1.
- Gradient descent will iteratively correct it, so it doesn't have to be perfect.
- The exact starting value of θ_1 usually doesn't matter for simple linear regression, as long as the learning rate α is appropriate and the cost function is convex.
- Gradient descent will converge to the global minimum.

Normalization in Linear Regression

- Suppose x values are very large, e.g., $x = 0\text{--}2000$:
- Even small errors $(h - y)$ are multiplied by a large number → θ_1 changes a lot in one step
- If learning rate α is not tiny → θ_1 can overshoot the minimum
- Cost may jump around which results in slow or unstable convergence

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

Normalization in Linear Regression

- Suppose x values are very large, e.g., $x = 0\text{--}2000$:
- Even small errors $(h - y)$ are multiplied by a large number $\rightarrow \theta_1$ changes a lot in one step
- If learning rate α is not tiny $\rightarrow \theta_1$ can overshoot the minimum
- Cost may jump around $\rightarrow$ slow or unstable convergence

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

Normalization Fixes It!!

- Scale x to a range like $0\text{--}1$ or mean 0 , std 1
- Now the update magnitude is consistent
- Gradient descent moves smoothly toward the minimum

Normalization in Linear Regression

- Suppose x values are very large, e.g., $x = 0\text{--}2000$:
- Even small errors ($h - y$) are multiplied by a large number $\rightarrow \theta_1$ changes a lot in one step
- If learning rate α is not tiny $\rightarrow \theta_1$ can overshoot the minimum
- Cost may jump around $\rightarrow$ slow or unstable convergence

Normalization Fixes It!!

- Scale x to a range like $0\text{--}1$ or mean 0 , std 1
- Now the update magnitude is consistent
- Gradient descent moves smoothly toward the minimum

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$
$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

gradient descent will still converge even without normalization, but it may be much slower because features with larger scales dominate the gradient, causing zig-zagging along steep directions.

normalization doesn't change whether convergence is possible, but it makes convergence faster, more stable, and easier to tune.

Min-Max Scaling

- Also known as feature scaling

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

After this transformation:

- The smallest value becomes 0
- The largest value becomes 1
- All other values are linearly mapped in between

When to Use Min-Max Scaling

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Sensitive to outliers!

- You need a **fixed bounded range** (e.g., [0, 1]) — often for neural networks with sigmoid/tanh activations or image pixel values.
- Outliers are not extreme or can be clipped.

Z-score normalization

- **Z-score normalization**, often simply called **standardization**, rescales data so that each value represents the number of standard deviations it lies away from the mean.

$$x' = \frac{x - \mu}{\sigma}$$

where:

- μ is the mean of the feature values
- σ is the standard deviation of the feature values

After standardization:

- The transformed data has mean 0
- The transformed data has standard deviation 1

When to Use Z-score Normalization

$$x' = \frac{x - \mu}{\sigma}$$

Features have very different scales or units

- Min-Max only rescales to $[0, 1]$, but doesn't equalize variance $\rightarrow$ can still cause slow or unstable gradient updates for uneven scales.
- Z-score both centers data around 0 and scales by standard deviation $\rightarrow$ variance of all features ≈ 1

Data contains outliers

- Z-score more robust to outliers also

Z score normalization

- Impact on bias term -

- When input features are centered around zero an additional and useful consequence appears:
 - The optimal value of bias term corresponds to the mean of the target variable

- $\bar{y} = \theta_0 + \theta_1 * \bar{x} \Rightarrow \text{where } (\bar{x} \approx 0)$
- $\bar{y} \approx \theta_0 + \theta_1 * 0 \Rightarrow$
- $\bar{y} \approx \theta_0 + 0$
- $\theta_0 \approx \bar{y}$

$$x' = \frac{x - \mu}{\sigma}$$

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Z score normalization

- Impact on bias term -

$$x' = \frac{x - \mu}{\sigma}$$

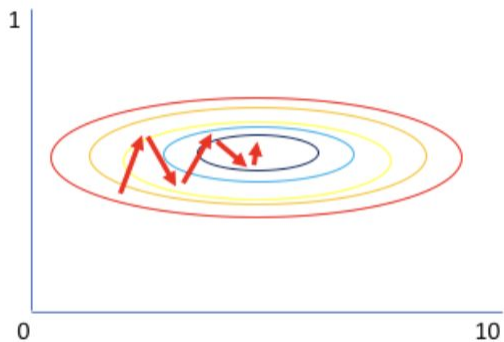
$$\theta_0 \approx \bar{y}$$

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

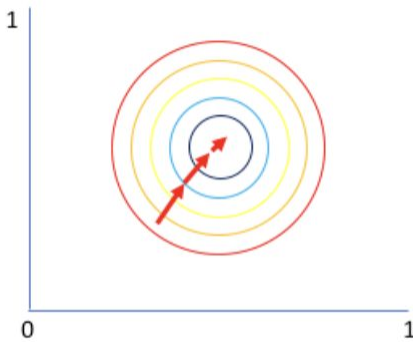
What This Means for Learning:

- You don't need to worry too much about tuning θ_0 , it will naturally settle at the mean of y .
- All the “interesting” learning happens in the slope parameters $\theta_1, \theta_2, \dots$ because they capture how y changes with x .

Contour Plot of Cost Surface



Gradient of larger parameter dominates the update



Both parameters can be updated in equal proportions

After normalization

- Cost changes in similar rates across all dimensions

Effect on gradient descent

- Direct path to minimum rather than a zigzag path
- Faster convergence
- Larger learning rates are safe

-

Making Prediction with Normalization

Assume our model is
 $y = 90 + 10x$

Suppose we are given a new input value representing an individual's age:

$$x_i = 20$$

Before making a prediction, we must **apply the same Z-score normalization** used during training:

$$x'_i = \frac{x_i - \mu_X}{\sigma_X}$$

where:

- μ_X is the mean of the training feature values
- σ_X is the standard deviation of the training feature values

If we have multiple features instead of just one slope parameter..

Multivariate Regression→ Linear regression with multiple input features

When we have multiple features, the hypothesis function can be extended as:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

where we have (n) different features.

We can simplify this notation using a summation:

$$h_{\theta}(x) = \theta_0 + \sum_{i=1}^n \theta_i x_i$$

Multivariate Regression

We can make it uniform by noting:

Thus, the hypothesis can be rewritten as:

Or, if we introduce a **dummy variable** $x_0 = 1$:

Which can be concisely written as:

$$h_{\theta}(x) = \theta_0 + \sum_{i=1}^n \theta_i x_i$$

$$\theta \cdot 1 = \theta$$

$$h_{\theta}(x) = \theta_0 \cdot 1 + \sum_{i=1}^n \theta_i x_i$$

$$h_{\theta}(x) = \theta_0 x_0 + \sum_{i=1}^n \theta_i x_i$$

$$h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i$$

Multivariate Cost Function and Gradient Descent

$$J(\theta) = \frac{1}{2m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Remember with 2 parameters

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^i$$

Repeat Until Convergence {

perform simultaneously {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}, \quad j = 1, 2, \dots, n$$

}}

θ_0 update is independent of features because it corresponds to the intercept (bias).

Its update depends only on the average error across all examples.

Polynomial Regression

- Regression where the model predicts y using powers of the feature:

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \dots + \theta_n x^n$$

- Still **linear in the parameters** $(\theta_0, \theta_1, \dots, \theta_n)$
- Non-linear in the **feature values** $(x, x^2, x^3, \dots)$

Polynomial Regression

- Why it is still Linear Regression??

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \dots + \theta_n x^n$$

- Each term multiplies a **parameter** (θ_j) by a known value (x^j)
- Gradient descent or normal equations work **exactly like linear regression**
- Trick: treat $x^2, x^3, \dots$ as **new features** $\rightarrow$ feature engineering

$$X_{\text{poly}} = [1, x, x^2, x^3, \dots, x^n]$$

Polynomial Regression

Why it is still Linear Regression?? EXAMPLE

- Original model (degree 3):

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

- Non-linear in feature x , but **linear in parameters** $\theta_0, \theta_1, \theta_2, \theta_3$

Map Polynomial Terms to "New Features"

- Let's define new variables:

$$a = x^2, \quad b = x^3$$

- Then the model becomes:

$$y = \theta_0 + \theta_1 x + \theta_2 a + \theta_3 b$$

Now it looks like a linear regression
with **3 features: x, a, b**

Polynomial Regression

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \dots + \theta_n x^n$$

- Polynomial regression = linear regression on transformed features
- Each new feature = power of original feature
- We can now use all standard linear regression tools (gradient descent, normal equation)

True Nonlinear regression

- True non-linear regression occurs when the parameters themselves appear non-linearly.
- Examples of non linear regression

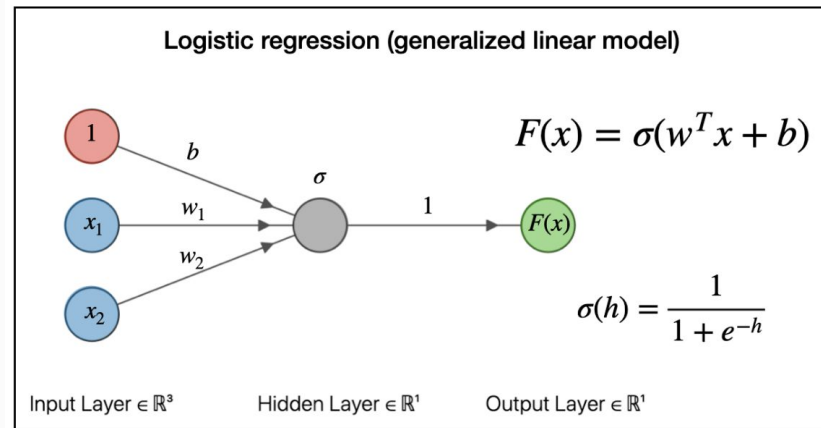
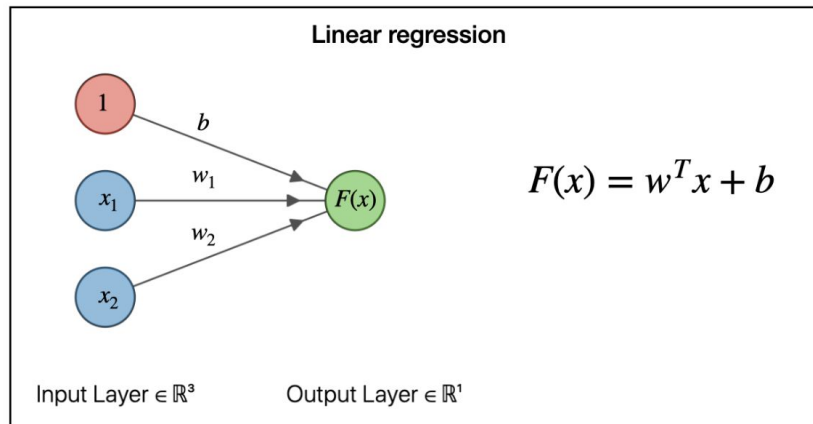
$$Y = \theta_0 + \theta_1^2 X$$

$$Y = \theta_0 e^{\theta_1 X}$$

$$Y = \theta_0 + \sin(\theta_1 X)$$

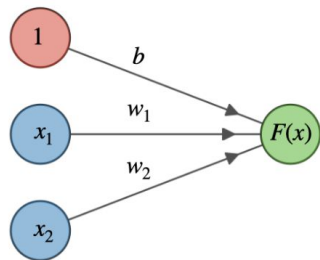
Neural networks perform non-linear regression, because parameters are inside non-linear activation functions!!

What's Next?



What's Near Future?

Linear regression

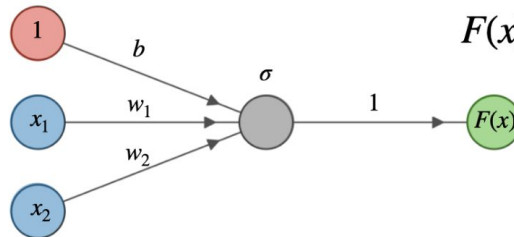


$$F(x) = w^T x + b$$

Input Layer $\in \mathbb{R}^3$

Output Layer $\in \mathbb{R}^1$

Logistic regression (generalized linear model)



$$F(x) = \sigma(w^T x + b)$$

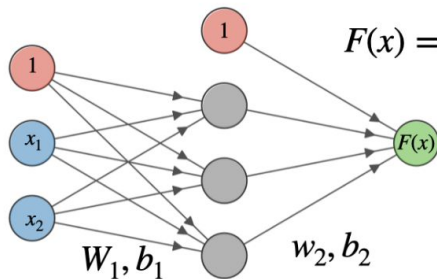
$$\sigma(h) = \frac{1}{1 + e^{-h}}$$

Input Layer $\in \mathbb{R}^3$

Hidden Layer $\in \mathbb{R}^1$

Output Layer $\in \mathbb{R}^1$

Shallow neural network (nonlinear regression)



$$F(x) = w_2^T \sigma(W_1^T x + b_1) + b_2$$

Input Layer $\in \mathbb{R}^3$

Hidden Layer $\in \mathbb{R}^4$

Output Layer $\in \mathbb{R}^1$

Project Proposal

- Expected to submit a project proposal by **Week-5 (25th of Feb, in~3 weeks)**
- A short one-pager with topic, dataset to use and team members
 - Will share project ideas, however feel free to pick one that interests you most
- Details will be shared within the week through email, meanwhile think about a problem and team up!

Lab Work

1. Download lecture notebook for week 2 and run it in Google Colab or on your own Jupyter server
2. Create your own data for the case of Day of Year vs Max Daily Temp
3. Run data through the workflow to (a) build model of data; and (b) visualise the data and the model.
4. Predict temperature for set of dates through the year.

APPENDIX

Making Predictions in Parallel

- Add Dummy Column for Intercept -

Scenario: Predict height from age

$$\text{height} = \theta_0 + \theta_1 \cdot \text{age}$$

- Feature vector (age for 5 people):

$$X = \begin{bmatrix} 10 \\ 15 \\ 20 \\ 25 \\ 30 \end{bmatrix}$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = \begin{bmatrix} 100 \\ 3 \end{bmatrix}$$

- `theta0 = 100` (intercept)
- `theta1 = 3` (slope)

Let's calculate predictions! We need to loop through X and call 'height' for each Xi

Making Predictions in Parallel

- Add Dummy Column for Intercept -

Scenario: Predict height from age

$$\text{height} = \theta_0 + \theta_1 \cdot \text{age}$$

- Feature vector (age for 5 people):

$$X = \begin{bmatrix} 10 \\ 15 \\ 20 \\ 25 \\ 30 \end{bmatrix}$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = \begin{bmatrix} 100 \\ 3 \end{bmatrix}$$

- `theta0 = 100` (intercept)
- `theta1 = 3` (slope)

- To include `theta0` (intercept) in **matrix multiplication**, add a column of 1s:

$$X = \begin{bmatrix} 1 & 10 \\ 1 & 15 \\ 1 & 20 \\ 1 & 25 \\ 1 & 30 \end{bmatrix}$$

$$y_{\text{pred}} = X \cdot \theta = \begin{bmatrix} 1 & 10 \\ 1 & 15 \\ 1 & 20 \\ 1 & 25 \\ 1 & 30 \end{bmatrix} \begin{bmatrix} 100 \\ 3 \end{bmatrix} = \begin{bmatrix} 130 \\ 145 \\ 160 \\ 175 \\ 190 \end{bmatrix}$$

- Each row = predicted height
- Computed **in parallel** for all ages

Let's calculate predictions! We need to loop through X and call 'height' for each Xi